

Django Models e BD



Prof. Dr. João Paulo Lemos Escola
Copyright© 2026

O que veremos?

- Introdução ao CRUD
- Definindo um Model
- Migrations e BD
- Django shell
- Listagem de registros no template.

Sumário

- 1) Models;
- 2) Manipulando sqlite com shell;
- 3) Mostrando dados na página;
- 4) Alterando o Model;
- 5) Publicando no GitHub.

Antes de começar...

- Baixe o projeto https://github.com/jpescola/my_tennis_club_aula1
- Descompacte o zip;
- Renomeie a pasta para **my_tennis_club_aula2**;
- Entre no cmd e digite os comandos:
 - `py -m venv .venv`
 - `.venv\scripts\activate`
 - `pip install django`
 - `py manage.py runserver`



1) Models

- Cada Model é uma tabela no banco de dados
- Até agora trabalhamos com páginas estáticas
- Agora vamos criar conteúdo dinâmico
- Primeiro passo para obter um CRUD

CRUD

- **Create, Read, Update, Delete**
- Criar um CRUD é criar um projeto que permita criar (cadastrar), ler (listar), atualizar e deletar registros no banco de dados;
- Django possui um ORM:
 - não é necessário criar classes ou métodos de manipulação de SQL.
- Basta programar os Models:
 - Eles serão responsáveis por manipular o SGBD.

Criando um model

- Abra a pasta do app;
- Adicione uma classe ao arquivo **models.py**

my_tennis_club/members/models.py:

```
from django.db import models

class Member(models.Model):
    firstname = models.CharField(max_length=255)
    lastname = models.CharField(max_length=255)
```

Vamos definir dois campos para o model Member.

Migration

- Após definir o Model, precisamos criar uma migração;
- Significa que o Django vai criar as tabelas no BD;
- Para isso, utilizaremos o comando makemigrations;
- Comando que cria a migração, ou seja:
 - Lê o arquivo models.py e gera o SQL

py manage.py makemigrations members

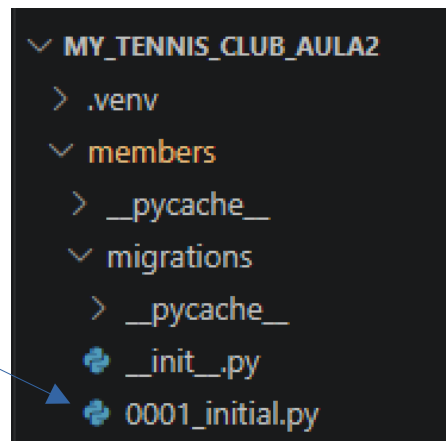
Veja que o arquivo de migração
0001 é criado na pasta
migrations do app members



```
Migrations for 'members':
  members\migrations\0001_initial.py
    - Create model Member
```

0001_initial.py

- Foi criado um arquivo python de migração;
- Esse arquivo contém os comandos SQL de criação das tabelas:



my_tennis_club/members/migrations/0001_initial.py:

Generated by Django 4.1.2 on 2022-10-27 11:14

from django.db import migrations, models

class Migration(migrations.Migration):

initial = True

dependencies = [
]

operations = [
 migrations.CreateModel(
 name='Member',
 fields=[
 ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
 ('firstname', models.CharField(max_length=255)),
 ('lastname', models.CharField(max_length=255)),
],
),
]

migrate

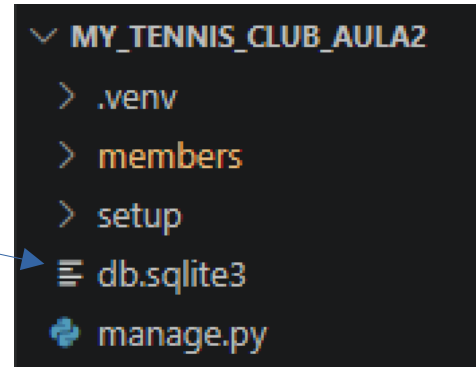
- Comando que executa as migrações pendentes;

py manage.py migrate

```
Operations to perform:  
  Apply all migrations: admin, auth, contenttypes, members, sessions  
Running migrations:  
  Applying members.0001_initial... OK
```

Sqlite

- Biblioteca que simula um SGBD
- Padrão em todo projeto Django
- Posteriormente aprenderemos a utilizar outros



Verificar o SQL gerado

py manage.py sqlmigrate members 0001

```
BEGIN;  
--  
-- Create model Member  
--  
CREATE TABLE "members_member" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "firstname" varchar(255) NOT NULL, "lastname" varchar(255) NOT NULL); COMMIT;
```

Resumo do Model

- Definir a classe (modelo) em models.py;
- Criar o migration:
 - makemigrations <app>
- Executar o migration:
 - migrate

2) Manipulando sqlite com shell

- Neste ponto, já temos o model “Member” declarado e as tabelas criadas no BD;
- Antes de continuar, vamos aprender a inserir dados manualmente no BD usando Django shell;
- Posteriormente vamos executar o projeto no servidor e visualizar o resultado no navegador.

Abrindo o shell

- `py manage.py shell`

```
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

Usando o shell

- Importando o model:

```
>>> from members.models import Member
```

- Listando os objetos (como um select):

```
>>> Member.objects.all()
```

- Nossa lista de membros está vazia:

```
<QuerySet []>
```

Inserindo um registro

- Para cadastrar um registro no banco, criamos um objeto Member e chamamos o método `save()`:

```
>>> member = Member(firstname='Emil', lastname='Refsnes')  
>>> member.save()
```

Listando os registros

- Para listar os registros do model Member:

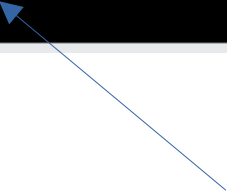
```
>>> Member.objects.all().values()
```

- Resultado da consulta:

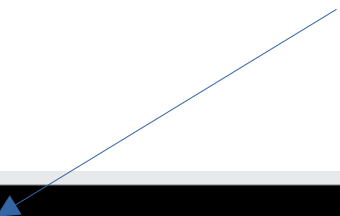
```
<QuerySet [{ 'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes' }]>
```

Inserindo múltiplos

```
>>> member1 = Member(firstname='Tobias', lastname='Refsnes')
>>> member2 = Member(firstname='Linus', lastname='Refsnes')
>>> member3 = Member(firstname='Lene', lastname='Refsnes')
>>> member4 = Member(firstname='Stale', lastname='Refsnes')
>>> member5 = Member(firstname='Jane', lastname='Doe')
>>> members_list = [member1, member2, member3, member4, member5]
>>> for x in members_list:
>>>     x.save()
```



Listando todos



```
>>> Member.objects.all().values()
<QuerySet [{ 'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes'},
{'id': 2, 'firstname': 'Tobias', 'lastname': 'Refsnes'},
{'id': 3, 'firstname': 'Linus', 'lastname': 'Refsnes'},
{'id': 4, 'firstname': 'Lene', 'lastname': 'Refsnes'},
{'id': 5, 'firstname': 'Stale', 'lastname': 'Refsnes'},
{'id': 6, 'firstname': 'Jane', 'lastname': 'Doe'}]>
```

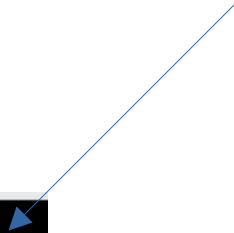
Atualizando registros

```
>>> from members.models import Member  
>>> x = Member.objects.all()[4]
```

```
>>> x.firstname
```

```
'Stale'
```

```
>>> x.firstname = "Stalikken"  
>>> x.save()
```



```
>>> Member.objects.all().values()
```

```
<QuerySet [{ 'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes'},  
{ 'id': 2, 'firstname': 'Tobias', 'lastname': 'Refsnes'},  
{ 'id': 3, 'firstname': 'Linus', 'lastname': 'Refsnes'},  
{ 'id': 4, 'firstname': 'Lene', 'lastname': 'Refsnes'},  
{ 'id': 5, 'firstname': 'Stalikken', 'lastname': 'Refsnes'},  
{ 'id': 6, 'firstname': 'Jane', 'lastname': 'Doe'}]>
```



Deletando registros

```
>>> from members.models import Member  
>>> x = Member.objects.all()[5]
```

```
>>> x.firstname 'Jane'
```

```
>>> x.delete() (1, {'members.Member': 1})
```



```
>>> Member.objects.all().values()  
<QuerySet [{'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes'},  
{'id': 2, 'firstname': 'Tobias', 'lastname': 'Refsnes'},  
{'id': 3, 'firstname': 'Linus', 'lastname': 'Refsnes'},  
{'id': 4, 'firstname': 'Lene', 'lastname': 'Refsnes'},  
{'id': 5, 'firstname': 'Stalikken', 'lastname': 'Refsnes'}]>
```

3) Mostrando dados na página

- Já temos um Model com dados fictícios;
- Agora vamos mostrá-los em uma página;
- Para isso, vamos configurar um ***template*** para mostrar um **vetor de membros**;
- Na ***view***, vamos **criar o vetor** a partir dos dados obtidos no BD.

Exibindo dados no template

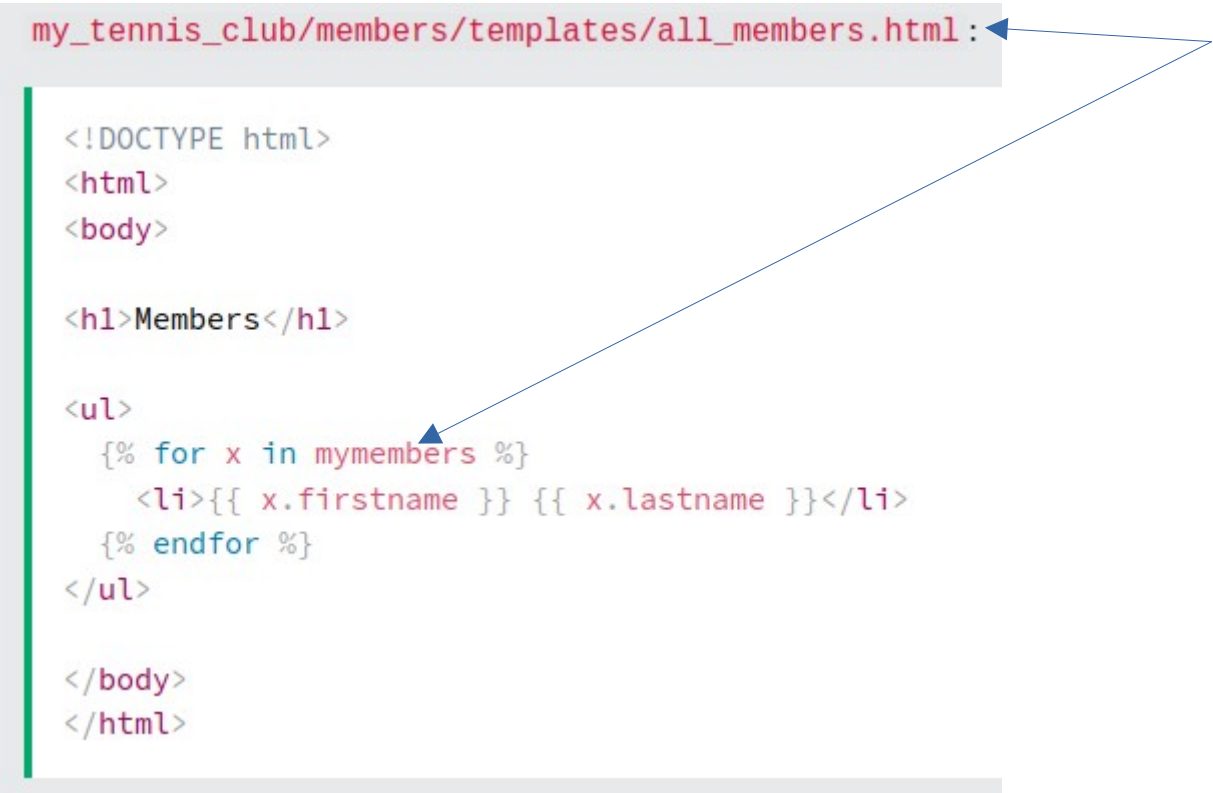
my_tennis_club/members/templates/all_members.html:

```
<!DOCTYPE html>
<html>
<body>

<h1>Members</h1>

<ul>
  {% for x in mymembers %}
    <li>{{ x.firstname }} {{ x.lastname }}</li>
  {% endfor %}
</ul>

</body>
</html>
```

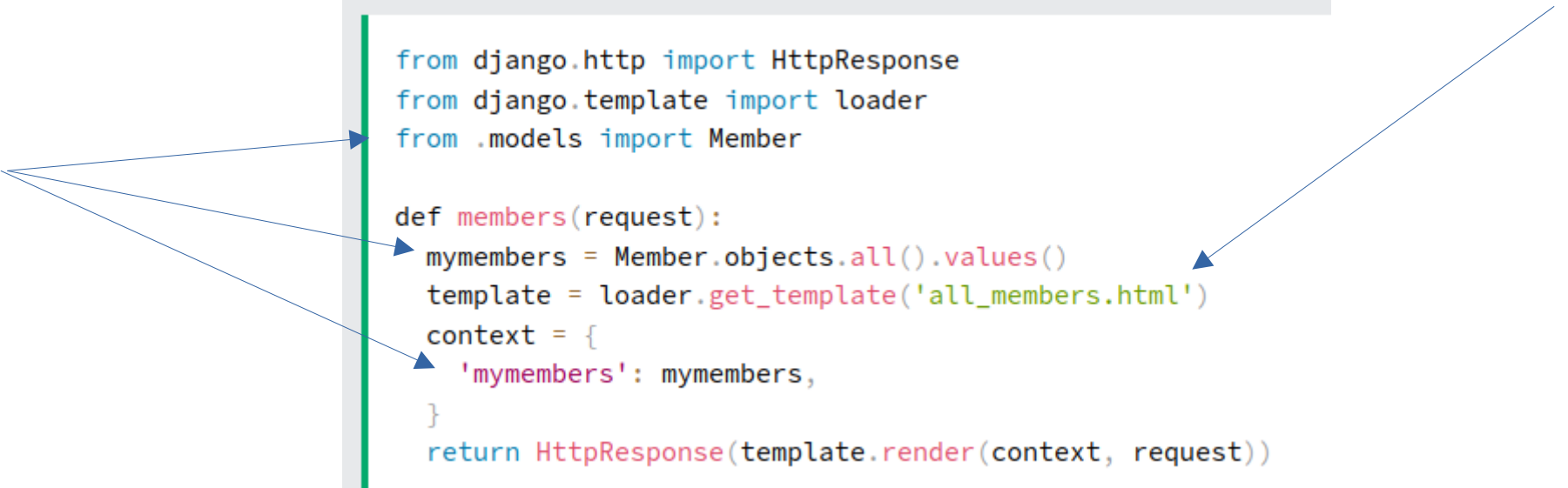


Alterando a view

my_tennis_club/members/views.py:

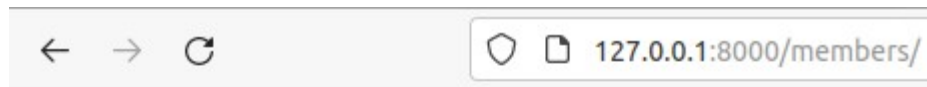
```
from django.http import HttpResponse
from django.template import loader
from .models import Member

def members(request):
    mymembers = Member.objects.all().values()
    template = loader.get_template('all_members.html')
    context = {
        'mymembers': mymembers,
    }
    return HttpResponse(template.render(context, request))
```



Visualizando o resultado

```
py manage.py runserver
```



Members

- Emil Refsnes
- Tobias Refsnes
- Linus Refsnes
- Lene Refsnes
- Stalikken Refsnes

4) Alteração de Model

- O que fazer quando precisamos alterar um Model? Exemplos:
 - Adicionar um novo campo ou corrigi-lo;
 - Remover um campo;

Resposta: Basta alterar o arquivo `models.py` e **refazer a migration.**

Alterando o Model

my_tennis_club/members/models.py:

```
from django.db import models

class Member(models.Model):
    firstname = models.CharField(max_length=255)
    lastname = models.CharField(max_length=255)
    phone = models.IntegerField()
    joined_date = models.DateField()
```

Não esqueça os parênteses

Vamos add dois campos.

```
py manage.py makemigrations members
```

Não permite campos nulos

- Não poderemos gerar a migração;
- Digite 2 e Enter:

```
py manage.py makemigrations members
You are trying to add a non-nullable field 'joined_date' to members without a default; we can't do that (the database needs something to
populate existing rows).
Please select a fix:
  1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
  2) Quit, and let me add a default in models.py
Select an option:
```

```
my_tennis_club/members/models.py :
```

```
from django.db import models

class Member(models.Model):
    firstname = models.CharField(max_length=255)
    lastname = models.CharField(max_length=255)
    phone = models.IntegerField(null=True)
    joined_date = models.DateField(null=True)
```

Permitir nulos.

```
py manage.py makemigrations members
```

```
Migrations for 'members':
  members\migrations\0002_member_joined_date_member_phone.py
    - Add field joined_date to member
    - Add field phone to member
```

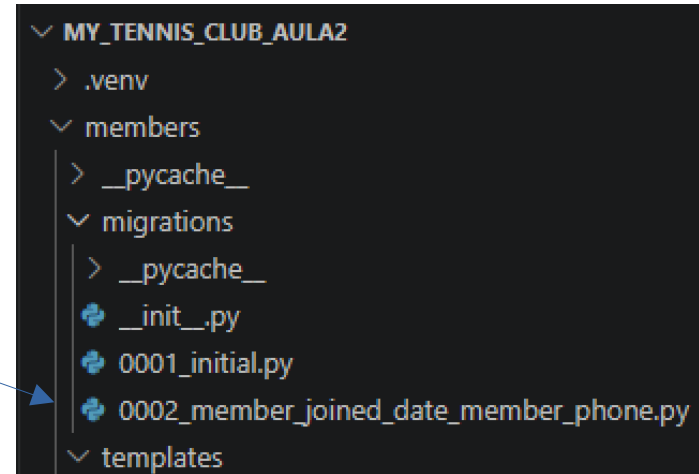
```
py manage.py migrate
```

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, members, sessions
Running migrations:
  Applying members.0002_member_joined_date_member_phone... OK

(myworld) C:\Users\Your Name\myworld\my_tennis_club>
```

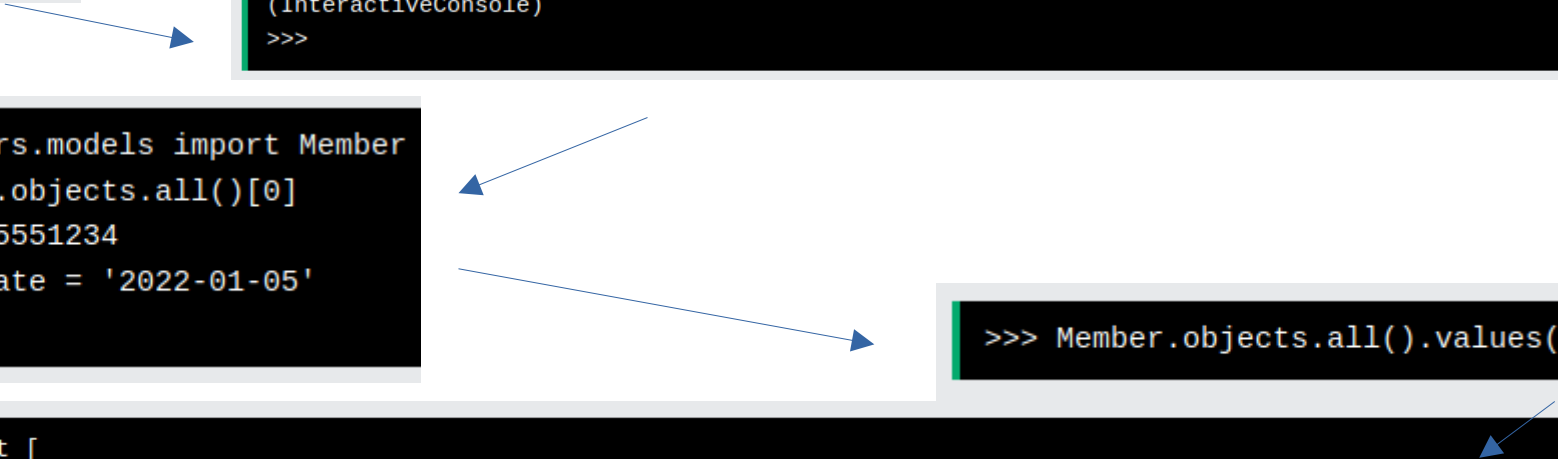
Nova migration foi gerada

- As migrações seguem uma numeração sequencial: 0001, 0002...



Atualizando dados

```
py manage.py shell
```



```
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
(InteractiveConsole)  
>>>
```

```
>>> from members.models import Member  
>>> x = Member.objects.all()[0]  
>>> x.phone = 5551234  
>>> x.joined_date = '2022-01-05'  
>>> x.save()
```

```
>>> Member.objects.all().values()
```

```
<QuerySet [  
{'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes', 'phone': 5551234, 'joined_date': datetime.date(2022, 1, 5)},  
{'id': 2, 'firstname': 'Tobias', 'lastname': 'Refsnes', 'phone': None, 'joined_date': None},  
{'id': 3, 'firstname': 'Linus', 'lastname': 'Refsnes', 'phone': None, 'joined_date': None},  
{'id': 4, 'firstname': 'Lene', 'lastname': 'Refsnes', 'phone': None, 'joined_date': None},  
{'id': 5, 'firstname': 'Stalikken', 'lastname': 'Refsnes', 'phone': None, 'joined_date': None}]>
```

Múltiplos registros fictícios

- Vamos cadastrar mais alguns registros fictícios usando a classe Faker;
- Com ela, podemos gerar dados falsos com aparência realística;
- Instale o pacote faker: `pip install faker`
- No django shell:

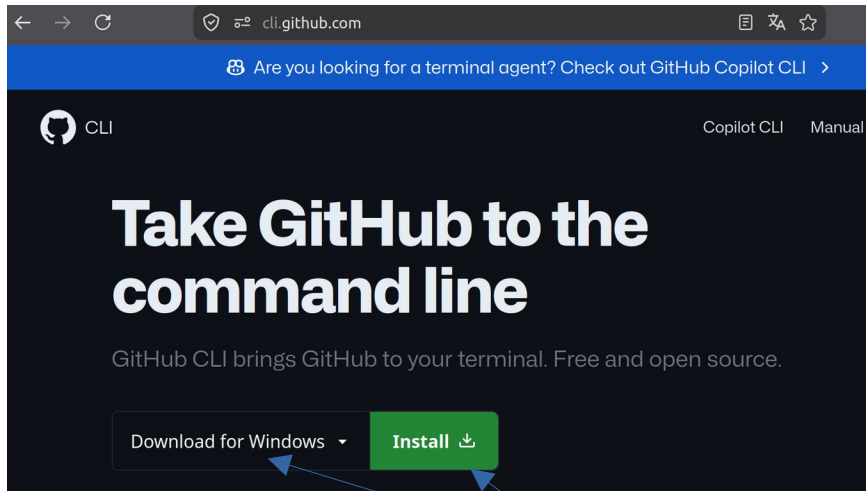
```
>>> from faker import Faker
>>> for _ in range(10): Member(firstname=Faker().first_name(), lastname=Faker().last_name()).save()
...
```



Pressione Enter 2x para confirmar

5) Publicando no GitHub

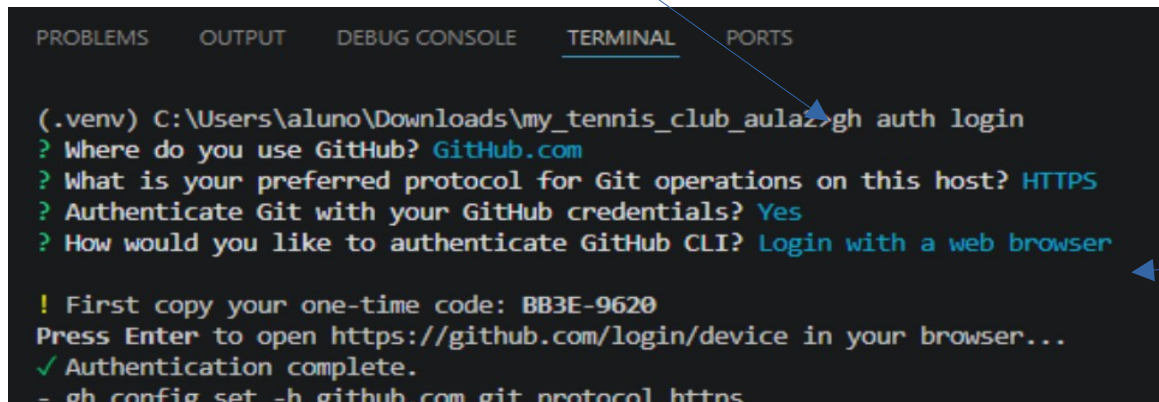
- Vamos aos passos para publicar um projeto de forma facilitada.



Instale o GitHub CLI em seu computador.

Abra o terminal

gh auth login



```
(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula>gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: BB3E-9620
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git protocol https
```

Pressione Enter nas 4 perguntas

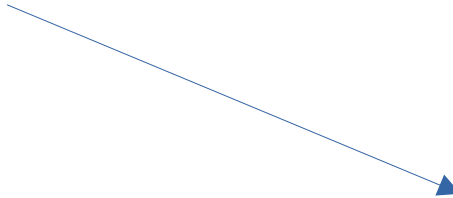
Surgirá um código de 8 caracteres e uma URL

Autorizando no GitHub

Acesse:

<https://github.com/login/device>

E digite o código do slide anterior



Authorize your device

 Signed in as **jpescola**

Enter the code displayed in the app or on the device you're signing in to.
Never use a code sent by someone else.

-

Continue

GitHub staff will never give you a code to enter on this page.



Authorize GitHub CLI

⚠ This authorization was requested from **Barretos 138.219.223.32** on July 24th, 2026 at 15:49 (-03)
Make sure you trust this device as it will get access to your account.



GitHub CLI by **GitHub**
wants to access your **jpescola** account

Existing access

- ✓ Create gists
- ✓ Read org and team membership, read org projects
- ✓ Full control of private repositories
- ✓ Update github action workflows

Organization access

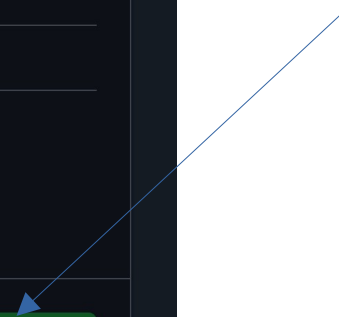
-  ads-brt ✓
-  NEXA-project-br ✓

Cancel

Authorize github

Requested from **Barretos 138.219.223.32** on July 24th, 2026 at 15:49 (-03)

Clique para
autorizar



Após autorizar no GitHub

- Pressione enter para confirmar:

```
! First copy your one-time code: BB3E-9620
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as jpescola

(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula2>
```

Agora o **gh** está conectado ao seu GitHub

Publicando o projeto atual

```
(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula2>gh repo create --public --source=. --push
✓ Created repository jpescola/my_tennis_club_aula2 on github.com
https://github.com/jpescola/my_tennis_club_aula2
✓ Added remote https://github.com/jpescola/my_tennis_club_aula2.git
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 4 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (34/34), 8.10 KiB | 592.00 KiB/s, done.
Total 34 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/jpescola/my_tennis_club_aula2.git
 * [new branch]      HEAD -> master
branch 'master' set up to track 'origin/master'.
✓ Pushed commits to https://github.com/jpescola/my_tennis_club_aula2.git

(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula2>
```

git init

git add .

git commit -m "primeiro commit"

gh repo create --public --source=. --push

O que vimos?

- Introdução ao CRUD
- Definindo um Model
- Migrations e BD
- Django shell
- Listagem de registros no template.